

Phase 5 Structured Note: Disjoint Sets and Graph Problem Solving

1. Current Learning Position

You have already learned the main graph foundations.

Previous phase	What you learned	Why it matters now
Phase 1	Graph vocabulary: vertex, edge, path, cycle, connected graph, directed graph, undirected graph	Disjoint Sets use these graph ideas, especially connectedness and cycles.
Phase 2	Graph representation: adjacency matrix, adjacency list, weighted graphs, compact lists, inverse lists	You know how graph edges can be stored in memory.
Phase 3	BFS: level-by-level graph traversal using a queue and visited array	BFS can discover connected vertices by moving through edges.
Phase 4	DFS: deep-first graph traversal using recursion or a stack and visited array	DFS can also discover connected vertices and reason about cycles.

Now Phase 5 introduces a new graph-related data structure: **Disjoint Sets**.

Disjoint Sets are also commonly called:

Disjoint Set Union
DSU
Union-Find

BFS and DFS answer questions like:

Starting from this vertex, what vertices can I reach?

Disjoint Sets answer a different question:

Are these two vertices already in the same connected group?

This is useful when we are adding edges one by one and want to know whether an edge creates a cycle.

2. What Is Covered in Phase 5?

In this phase, you will learn:

1. What disjoint subsets are
 2. Why Disjoint Sets are useful in graph problems
 3. Array representation of sets
 4. Negative values for roots and set sizes
 5. Positive values as parent pointers
 6. The `find` operation
 7. The `union` operation
 8. Weighted union
 9. Cycle detection using `find` before `union`
 10. Connection to Kruskal's Algorithm
 11. BFS vs DFS vs Disjoint Sets
 12. Complete C++ program for cycle detection
 13. Program dry run
 14. Time and space complexity
 15. Common beginner mistakes
 16. Final checklist
-

3. Big Idea of Disjoint Sets

A **set** is a group of elements.

A **disjoint set** means a group that does not overlap with another group.

Simple example:

```
Set A = {1, 2, 3}
Set B = {4, 5}
```

These two sets are disjoint because they have no common element.

There is no number that appears in both Set A and Set B.

So:

```
A ∩ B = empty
```

Simple meaning:

Disjoint sets are separate groups.
Each element belongs to only one group.

4. Beginner Intuition

Imagine students are split into project teams.

At first, every student is alone:

$\{1\}$ $\{2\}$ $\{3\}$ $\{4\}$ $\{5\}$

If student 1 joins student 2's team:

$\{1, 2\}$ $\{3\}$ $\{4\}$ $\{5\}$

If student 3 joins that same team:

$\{1, 2, 3\}$ $\{4\}$ $\{5\}$

Now if someone asks:

Are student 1 and student 3 in the same team?

The answer is:

Yes.

If someone asks:

Are student 1 and student 5 in the same team?

The answer is:

No.

Disjoint Sets help a program answer this kind of question quickly.

5. Why Disjoint Sets Matter in Graphs

In a graph, vertices can form connected groups.

Example:

```
1 ---- 2 ---- 3      4 ---- 5
```

This graph has two connected groups:

```
{1, 2, 3}
{4, 5}
```

Disjoint Sets can store these connected groups.

They help us answer:

```
Are vertex 1 and vertex 3 already connected?
```

Yes.

```
Are vertex 1 and vertex 5 already connected?
```

No.

This is especially useful when edges are added one by one.

6. Disjoint Sets vs BFS and DFS

BFS and DFS travel through the graph.

Disjoint Sets do not travel through the graph in the same way.

BFS and DFS are traversal algorithms.

Disjoint Set Union is a grouping data structure.

Tool	What it does
BFS	Visits vertices level by level
DFS	Visits vertices deeply and backtracks
Disjoint Set	Tracks which vertices belong to the same group

Simple memory line:

```
BFS visits.
DFS dives.
Union-Find groups.
```

7. The Main Operations

Disjoint Sets mainly use two operations:

Operation	Simple meaning
<code>find(x)</code>	Find the root or leader of the group containing <code>x</code>
<code>union(a, b)</code>	Merge the groups containing <code>a</code> and <code>b</code>

Because of these two operations, Disjoint Sets are often called **Union-Find**.

8. What Is a Root?

Each group has one special representative.

That representative is called the **root**.

Example:

```
Group: {3, 5, 9}
Root: 3
```

This means:

3 represents the whole group.
5 belongs to 3's group.
9 belongs to 3's group.

If we ask:

find(5)

The answer should be:

3

If we ask:

find(9)

The answer should also be:

3

Because both 5 and 9 belong to the group whose root is 3.

9. Array Representation of Disjoint Sets

Disjoint Sets can be stored using one array.

The array is often called:

parent

or sometimes:

s

For example:

```
vector<int> parent(n + 1, -1);
```

If there are 5 vertices, the initial array is:

```
parent[1] = -1
parent[2] = -1
parent[3] = -1
parent[4] = -1
parent[5] = -1
```

Why all `-1`?

Because at the beginning, every vertex is alone in its own set.

So each vertex is the root of a set of size 1.

10. Meaning of Negative and Positive Values

The parent array uses two types of values.

Array value	Meaning
Negative value	This index is a root
Positive value	This index points to another parent/root

Examples:

Value	Simple meaning
<code>-1</code>	Root of a set of size 1
<code>-3</code>	Root of a set of size 3
<code>-7</code>	Root of a set of size 7
<code>4</code>	This vertex points to vertex 4 as its parent

Important idea:

```
Negative = root and size
Positive = parent pointer
```

11. Why Do Roots Store Negative Sizes?

A root stores a negative number because the negative sign tells us:

This is a root.

The size tells us:

How many elements are in this set.

Example:

```
parent[4] = -7
```

This means:

4 is the root.
The set containing 4 has 7 elements.

The actual size is 7.

The negative sign is not an error.

It is part of the representation.

12. Example of Parent Array Representation

Suppose we have this set:

```
Set A = {3, 5, 9}  
Root = 3
```

The array can store this as:

```
parent[3] = -3  
parent[5] = 3  
parent[9] = 3
```


Meaning:

```
3 is the root of a set of size 3.  
5 points to 3.  
9 points to 3.
```

Now suppose we have another set:

```
Set B = {4, 7, 8, 10}  
Root = 4
```

The array can store this as:

```
parent[4] = -4  
parent[7] = 4  
parent[8] = 4  
parent[10] = 4
```

Meaning:

```
4 is the root of a set of size 4.  
7 points to 4.  
8 points to 4.  
10 points to 4.
```

13. Visual Meaning of Parent Pointers

The array:

```
parent[3] = -3  
parent[5] = 3  
parent[9] = 3
```

Can be imagined like this:

```
  3
 /
5  9
```

3 is the root.

5 and 9 point to 3.

The value `-3` at index 3 means:

This root represents 3 elements.

14. The `find` Operation

The `find` operation answers this question:

What is the root of the set containing this element?

Example:

```
find(9)
```

Means:

Which group does 9 belong to?
Who is the root of that group?

15. How `find` Works

The rule is:

Start at the element.
Follow parent pointers.
Stop when you reach a negative value.
That index is the root.

Why stop at a negative value?

Because a negative value means:

This index is a root.

16. Find Example

Suppose the parent array contains:

```
parent[9] = 3  
parent[3] = 4  
parent[4] = -7
```

Now run:

```
find(9)
```

Step-by-step:

Step	Current value	Array value	Meaning
Start	x = 9	parent[9] = 3	9 points to 3
Next	x = 3	parent[3] = 4	3 points to 4
Next	x = 4	parent[4] = -7	4 is a root

So:

```
find(9) = 4
```

The root of 9 is 4.

17. Find Code

```
int findRoot(vector<int>& parent, int u) {  
    int x = u;
```

```
while (parent[x] > 0) {  
    x = parent[x];  
}  
  
return x;  
}
```

18. Find Code Explanation

Function header

```
int findRoot(vector<int>& parent, int u)
```

This function receives:

Parameter	Meaning
<code>parent</code>	The array storing the sets
<code>u</code>	The vertex whose root we want to find

It returns an integer:

The root of u

Start from `u`

```
int x = u;
```

This means:

Start searching from the given vertex.

Follow parent pointers

```
while (parent[x] > 0) {  
    x = parent[x];  
}
```

This means:

As long as x is not a root, move upward to its parent.

Remember:

Positive value = parent pointer
Negative value = root

Return the root

```
return x;
```

When the loop stops, `x` is the root.

19. The `union` Operation

The `union` operation merges two sets.

Example:

```
Set A = {1, 2}  
Set B = {3, 4}
```

After union:

```
{1, 2, 3, 4}
```

Simple meaning:

Union combines two separate groups into one group.

20. Why We Should Find Roots Before Union

Before merging two sets, we should find their roots.

Wrong idea:

```
weightedUnion(parent, u, v);
```

This is risky because `u` and `v` may not be roots.

Correct idea:

```
int rootU = findRoot(parent, u);  
int rootV = findRoot(parent, v);  
weightedUnion(parent, rootU, rootV);
```

Why?

Because only roots store correct negative size values.

Normal vertices store parent pointers, not set sizes.

21. Weighted Union

Weighted union is a smarter way to merge sets.

It says:

```
Attach the smaller set under the larger set.
```

Why?

Because this keeps the tree shorter.

Shorter trees make future `find` operations faster.

22. How to Compare Set Sizes

Roots store negative sizes.

Example:

```
parent[3] = -3  
parent[4] = -4
```

This means:

```
Root 3 has set size 3.  
Root 4 has set size 4.
```

Which set is larger?

Root 4's set is larger.

Why?

Because `-4` represents size 4.

Important comparison:

```
-4 is more negative than -3.  
The more negative value represents the larger set.
```

23. Weighted Union Dry Run

Suppose:

```
parent[3] = -3  
parent[4] = -4
```

We want to union the sets whose roots are 3 and 4.

Step 1: Compare sizes

```
Set rooted at 3 has size 3.  
Set rooted at 4 has size 4.
```

The set rooted at 4 is larger.

Step 2: Combine sizes

```
parent[4] = -3 + -4  
parent[4] = -7
```

Step 3: Make smaller root point to larger root

```
parent[3] = 4
```

Final result:

```
parent[4] = -7  
parent[3] = 4
```

Meaning:

```
4 is now the root of the combined set.  
The combined set has size 7.  
3 is no longer a root.  
3 now points to 4.
```

24. Weighted Union Code

```
void weightedUnion(vector<int>& parent, int rootU, int rootV) {  
    if (parent[rootU] < parent[rootV]) {  
        parent[rootU] = parent[rootU] + parent[rootV];  
        parent[rootV] = rootU;  
    } else {  
        parent[rootV] = parent[rootU] + parent[rootV];  
        parent[rootU] = rootV;  
    }  
}
```

25. Weighted Union Code Explanation

Function header

```
void weightedUnion(vector<int>& parent, int rootU, int rootV)
```

This function receives:

Parameter	Meaning
<code>parent</code>	The disjoint set array
<code>rootU</code>	Root of the first set
<code>rootV</code>	Root of the second set

Important:

`rootU` and `rootV` must be roots.

If `rootU`'s set is larger

```
if (parent[rootU] < parent[rootV])
```

Because sizes are negative, the more negative value means the larger set.

Example:

`-5 < -2`

So a root with `-5` represents a larger set than a root with `-2`.

Combine under `rootU`

```
parent[rootU] = parent[rootU] + parent[rootV];
parent[rootV] = rootU;
```

This means:

Add the sizes together.
Make `rootV` point to `rootU`.

Otherwise combine under `rootV`

```
parent[rootV] = parent[rootU] + parent[rootV];
parent[rootU] = rootV;
```

This means:

Add the sizes together.
Make rootU point to rootV.

26. Cycle Detection in an Undirected Graph

Disjoint Sets are often used to detect cycles in an undirected graph.

The question is:

If I add this edge, will it create a cycle?

The rule is simple:

```
For edge u--v:  
rootU = find(u)  
rootV = find(v)
```

If $\text{rootU} == \text{rootV}$, adding $u--v$ creates a cycle.
If $\text{rootU} != \text{rootV}$, adding $u--v$ is safe, so union the sets.

27. Why $\text{find}(u) == \text{find}(v)$ Means Cycle

If $\text{find}(u) == \text{find}(v)$, then u and v already belong to the same connected group.

That means there is already some path between u and v .

If we add another edge directly between them, we create a closed loop.

A closed loop is a cycle.

Example:

2 ---- 3 ---- 4 ---- 5

There is already a path from 2 to 5:

```
2 -> 3 -> 4 -> 5
```

Now add edge:

```
5 ---- 2
```

This creates:

```
2 -> 3 -> 4 -> 5 -> 2
```

That is a cycle.

28. Cycle Detection Table

Situation	Meaning	Action
<code>find(u) != find(v)</code>	<code>u</code> and <code>v</code> are in different groups	Add edge and union the groups
<code>find(u) == find(v)</code>	<code>u</code> and <code>v</code> are already connected	Adding the edge creates a cycle

This is one of the most important ideas in Phase 5.

29. Function to Add Edge and Check Cycle

```
bool addEdgeAndCheckCycle(vector<int>& parent, int u, int v) {
    int rootU = findRoot(parent, u);
    int rootV = findRoot(parent, v);

    if (rootU == rootV) {
        return true;
    }

    weightedUnion(parent, rootU, rootV);
    return false;
}
```

30. Function Explanation

Find the roots

```
int rootU = findRoot(parent, u);  
int rootV = findRoot(parent, v);
```

This finds the group leaders of `u` and `v`.

Check whether they are already connected

```
if (rootU == rootV) {  
    return true;  
}
```

If both roots are the same, `u` and `v` are already in the same set.

So adding the edge creates a cycle.

If no cycle, merge the sets

```
weightedUnion(parent, rootU, rootV);  
return false;
```

If the roots are different, the edge is safe.

So we merge the two sets.

31. Complete Phase 5 C++ Program

```
#include <iostream>  
#include <vector>  
using namespace std;  
  
int findRoot(vector<int>& parent, int u) {  
    int x = u;  
  
    while (parent[x] > 0) {  
        x = parent[x];  
    }  
}
```

```

        return x;
    }

    void weightedUnion(vector<int>& parent, int rootU, int rootV) {
        // Roots store negative sizes.
        // More negative means the set is larger.
        if (parent[rootU] < parent[rootV]) {
            parent[rootU] = parent[rootU] + parent[rootV];
            parent[rootV] = rootU;
        } else {
            parent[rootV] = parent[rootU] + parent[rootV];
            parent[rootU] = rootV;
        }
    }

    bool addEdgeAndCheckCycle(vector<int>& parent, int u, int v) {
        int rootU = findRoot(parent, u);
        int rootV = findRoot(parent, v);

        if (rootU == rootV) {
            return true; // cycle detected
        }

        weightedUnion(parent, rootU, rootV);
        return false; // no cycle
    }

    int main() {
        int n = 5;

        vector<pair<int, int>> edges = {
            {1, 2},
            {2, 3},
            {3, 4},
            {4, 5},
            {5, 2}
        };

        vector<int> parent(n + 1, -1);

        for (auto edge : edges) {
            int u = edge.first;
            int v = edge.second;

            cout << "Adding edge " << u << "--" << v << ": ";

            if (addEdgeAndCheckCycle(parent, u, v)) {
                cout << "cycle detected\n";
            }
        }
    }

```

```
        } else {
            cout << "safe, sets merged\n";
        }
    }

    cout << "\nFinal parent array:\n";
    for (int i = 1; i <= n; i++) {
        cout << "parent[" << i << "] = " << parent[i] << "\n";
    }

    return 0;
}
```

32. Expected Output Idea

```
Adding edge 1--2: safe, sets merged
Adding edge 2--3: safe, sets merged
Adding edge 3--4: safe, sets merged
Adding edge 4--5: safe, sets merged
Adding edge 5--2: cycle detected
```

The exact final parent array may depend on how ties are handled in weighted union.

But the important result is:

```
The edge 5--2 creates a cycle.
```

33. Program Dry Run

Edges:

```
1--2
2--3
3--4
4--5
5--2
```

Initial parent array:

```
parent[1] = -1
parent[2] = -1
parent[3] = -1
parent[4] = -1
parent[5] = -1
```

Every vertex is alone.

34. Edge 1--2

Find roots:

```
find(1) = 1
find(2) = 2
```

The roots are different.

So edge `1--2` is safe.

Union the sets:

```
{1} and {2} become one set.
```

Now 1 and 2 are connected.

35. Edge 2--3

Find roots:

```
find(2) = root of the group containing 2
find(3) = 3
```

2 is already connected to 1.

3 is alone.

The roots are different.

So edge `2--3` is safe.

Union the sets:

$\{1, 2\}$ and $\{3\}$ become one set.

Now the connected group is:

$\{1, 2, 3\}$

36. Edge 3--4

Find roots:

$\text{find}(3) = \text{root of } \{1, 2, 3\}$
 $\text{find}(4) = 4$

The roots are different.

So edge $3-4$ is safe.

Union the sets:

$\{1, 2, 3\}$ and $\{4\}$ become one set.

Now the connected group is:

$\{1, 2, 3, 4\}$

37. Edge 4--5

Find roots:

$\text{find}(4) = \text{root of } \{1, 2, 3, 4\}$
 $\text{find}(5) = 5$

The roots are different.

So edge 4--5 is safe.

Union the sets:

{1, 2, 3, 4} and {5} become one set.

Now the connected group is:

{1, 2, 3, 4, 5}

38. Edge 5--2

Find roots:

find(5) = root of {1, 2, 3, 4, 5}
find(2) = root of {1, 2, 3, 4, 5}

The roots are the same.

That means 5 and 2 are already connected.

There is already a path:

2 ---- 3 ---- 4 ---- 5

If we add edge:

5 ---- 2

We create a cycle:

```
2 ---- 3 ---- 4 ---- 5
|                               |
+-----+
```

So the program prints:

cycle detected

39. Summary of the Dry Run

Edge	Are roots different?	Action
1--2	Yes	Safe, merge sets
2--3	Yes	Safe, merge sets
3--4	Yes	Safe, merge sets
4--5	Yes	Safe, merge sets
5--2	No	Cycle detected

40. Connection to Kruskal's Algorithm

Disjoint Sets are very important in **Kruskal's Algorithm**.

Kruskal's Algorithm is used to build a **Minimum Spanning Tree**.

A spanning tree connects all vertices without cycles.

Kruskal's Algorithm considers edges one by one.

For each edge, it asks:

Would adding this edge create a cycle?

Disjoint Sets answer that question quickly.

Rule used in Kruskal's Algorithm:

If `find(u) == find(v)`, skip the edge because it creates a cycle.
If `find(u) != find(v)`, accept the edge and union the sets.

Phase 5 does not fully teach Kruskal's Algorithm, but it prepares you for it.

41. BFS vs DFS vs Disjoint Set

Tool	Main question answered	Memory tool	Typical use
BFS	What vertices are reachable level by level from a start?	Queue + visited array	Shortest unweighted path levels, reachability, connected components
DFS	How deep can I explore before backtracking?	Recursion/system stack + visited array	Traversal, cycle ideas, components, topological-style reasoning
Disjoint Set	Are two vertices already in the same connected group?	Parent/size array	Cycle detection while adding edges, Kruskal-style MST preparation

42. Important Difference Between Traversal and Union-Find

BFS and DFS physically move through graph edges.

They can print an order like:

1 2 3 4 5

Disjoint Set does not normally print a traversal order.

It does not ask:

Where should I visit next?

It asks:

Which group does this vertex belong to?

and:

Are these two vertices in the same group?

43. Time Complexity

For the simple version shown in this phase:

find can take up to $O(N)$ in the worst case.

Why?

Because `find` may need to follow parent pointers up a chain.

Weighted union helps keep chains shorter than a naive union.

So it improves performance in practice.

A more advanced version usually adds **path compression**, but that optimization is not part of this phase.

44. Space Complexity

The Disjoint Set structure uses one parent array.

If there are `N` vertices:

parent array size = `N`

So the space complexity is:

$O(N)$

or:

$O(V)$

where `V` is the number of vertices.

45. Common Beginner Mistakes

Mistake 1: Thinking negative values are errors

Wrong idea:

```
parent[4] = -7 must be wrong.
```

Correct idea:

```
A negative value means this index is a root.  
The size of the set is 7.
```

Mistake 2: Calling union on normal vertices instead of roots

Wrong:

```
weightedUnion(parent, u, v);
```

Correct:

```
int rootU = findRoot(parent, u);  
int rootV = findRoot(parent, v);  
weightedUnion(parent, rootU, rootV);
```

Only roots store valid negative sizes.

Mistake 3: Forgetting to check for a cycle before union

Wrong order:

```
Union first, then check.
```

Correct order:

```
Find roots first.  
If roots are the same, cycle detected.  
If roots are different, union them.
```

Mistake 4: Confusing Disjoint Set with BFS or DFS

Wrong idea:

```
Union-Find should print vertices in visit order.
```

Correct idea:

```
Union-Find does not traverse like BFS or DFS.  
It tracks connected groups.
```

Mistake 5: Comparing parent values of non-root vertices

Wrong:

```
if (parent[u] < parent[v])
```

This is only meaningful if `u` and `v` are roots.

Correct:

```
int rootU = findRoot(parent, u);  
int rootV = findRoot(parent, v);
```

Then compare:

```
parent[rootU]  
parent[rootV]
```

Mistake 6: Forgetting the cycle rule

Important rule:

If `find(u) == find(v)`, adding edge `u--v` creates a cycle.

This is the heart of cycle detection using Disjoint Sets.

46. Memory Tricks

Trick 1

Find = find the leader.
Union = join two groups.

Trick 2

Negative means root.
Positive means pointer.

Trick 3

Same root means same group.
Same group plus new edge means cycle.

Trick 4

BFS visits.
DFS dives.
Union-Find groups.

47. Phase 5 Summary Table

Concept	Simple meaning
Disjoint sets	Separate groups with no overlapping elements
Union-Find	Another name for Disjoint Set Union

Concept	Simple meaning
Parent array	Array used to store set roots and parent links
Root	The representative or leader of a set
Negative parent value	Means this index is a root and stores set size
Positive parent value	Means this index points to another parent/root
<code>find(x)</code>	Finds the root of the set containing <code>x</code>
<code>union(a, b)</code>	Merges two sets
Weighted union	Attaches smaller set under larger set
Cycle detection	If two vertices already have the same root, adding their edge creates a cycle
Kruskal context	Union-Find helps Kruskal avoid cycles while building a spanning tree

48. Mini Practice Questions

Question 1

What does it mean for two sets to be disjoint?

Answer:

They have no elements in common.

Question 2

What does this mean?

`parent[4] = -5`

Answer:

4 is the root of a set of size 5.

Question 3

What does this mean?

```
parent[6] = 4
```

Answer:

```
6 points to 4 as its parent.  
6 is not a root.
```

Question 4

What does `find(x)` do?

Answer:

```
It follows parent pointers until it finds the root of x's set.
```

Question 5

Why do we call `find` before `union`?

Answer:

```
Because union should merge roots, not ordinary vertices.  
Only roots store correct set sizes.
```

Question 6

Given:

```
parent[9] = 3  
parent[3] = 4  
parent[4] = -7
```

What is `find(9)` ?

Answer:

`find(9) = 4`

Because:

`9 -> 3 -> 4`

and 4 is the root.

Question 7

If `find(u) == find(v)`, what happens when we add edge `u--v` ?

Answer:

A cycle is created.

Question 8

If `find(u) != find(v)`, what should we do?

Answer:

The edge is safe, so we union the two sets.

Question 9

What is weighted union?

Answer:

Weighted union attaches the smaller set under the larger set.

Question 10

Is Union-Find a traversal algorithm like BFS or DFS?

Answer:

No.
Union-Find tracks connected groups.
It does not visit vertices in traversal order.

49. Practice Dry Run

Edges:

1--2
3--4
2--3
4--1

Start:

{1} {2} {3} {4}

Add 1--2

```
find(1) != find(2)
```

Safe.

After union:

{1, 2} {3} {4}

Add 3--4

```
find(3) != find(4)
```

Safe.

After union:

```
{1, 2} {3, 4}
```

Add 2--3

```
find(2) != find(3)
```

Safe.

After union:

```
{1, 2, 3, 4}
```

Add 4--1

```
find(4) == find(1)
```

They are already in the same group.

So adding `4--1` creates a cycle.

Final result:

```
Cycle detected at edge 4--1.
```

50. Final Phase 5 Checklist

You understand Phase 5 if you can explain these without looking:

- What disjoint sets are
- What Union-Find means
- Why Disjoint Sets are useful in graph problems
- How the parent array stores sets
- Why every vertex starts with `-1`
- What a root is
- Why root values are negative
- Why the negative value stores set size
- What positive parent values mean
- How `find` follows parent pointers
- How `union` merges two sets
- Why weighted union attaches smaller sets under larger sets
- Why you must call `find` before `union`

- How `find(u) == find(v)` detects a cycle
 - How Disjoint Sets differ from BFS and DFS
 - How Disjoint Sets prepare you for Kruskal's Algorithm
 - How to dry run cycle detection edge by edge
 - The common beginner mistakes
-

51. Final Phase 5 Takeaway

Disjoint Set Union is a data structure that tracks connected groups.

It is useful when edges are added one by one.

For each edge `u--v`, ask:

Are u and v already in the same group?

Use:

```
find(u)
find(v)
```

If both roots are the same:

Adding the edge creates a cycle.

If the roots are different:

The edge is safe.
Merge the groups using union.

The core pattern is:

```
rootU = find(u)
rootV = find(v)

if rootU == rootV:
    cycle detected
else:
    union(rootU, rootV)
```

That is the heart of Phase 5.